European Flight Arrival Efficiency, Fuel Penalties & Machine Learning

An End-to-End Medallion PySpark Pipeline on EUROCONTROL OPDI Data

Mathis Nancey & **Thomas Canisares–Morère**
Group 08 | Course: DAT535 Data Engineering
Department of Electrical Engineering and Computer Science, University of Stavanger (UiS)

Production Run: `prod-8` (2026-11-09) | Academic Year 2026

Abstract

This report documents a **PySpark 3.5** pipeline structured around the **Medallion Architecture** (Bronze, Silver, Gold, plus an ML layer). Over **58 days** (June 1 – July 28, 2026) of **EUROCONTROL OPDI v0.0.2** data and the **OurAirports** reference data, the pipeline processes **3,304,717 flights**, **86,670,133 flight events** and **118,779,222 measurements** (5.46 GB in Bronze Parquet).

The served study measures the arrival efficiency at Paris-CDG, Toulouse and Stavanger, compared with London-Heathrow and Frankfurt: additional transit time from a 100 NM ring to the runway (Indicator A), level-offs and holdings during descent (Indicator B). In production run `prod-8`, **Heathrow** has the highest additional time of the five airports (4.9 min, measured on 14 % of its arrivals) and 12.7 % of its arrivals hold; its additional time grows with demand, from about 0.2 min below 20 arrivals per hour to 6.8 min at 40–49 arrivals per hour. **Paris-CDG** and **Frankfurt** rarely hold (2.9 %) but fly 4.6 and 4.1 minutes level per arrival, i.e. about 30 and 34 tonnes of fuel per day under a 12.7 kg/min hypothesis. **Toulouse** is fluid (1.9 min, 1.6 min of level flight); at **Stavanger** the runway coverage is too low to measure Indicator A. A Spark MLlib `GBTRegressor` beats a naive baseline (test MAE 2.74 vs 2.84 min, R^2 0.135 vs 0.053); the destination airport is by far its most important variable.

1 Project Statement and Scope

Project objective. The project builds an automated and reproducible pipeline that ingests the raw OPDI flight list, flight events and measurements, turns them into trusted and reusable Silver tables, and serves Gold tables for one analytical question: *how efficient is the arrival phase at a given airport, and when and where is time (and fuel) lost?* The consumers are the analysis notebooks of the repository (Gold dashboard, arrival-efficiency study) and the ML layer.

Operational problem. When the demand approaches the runway capacity, air traffic control absorbs the delay in the air: **holdings** (racetrack patterns flown at a fixed level while waiting for the approach) and **level-offs** that interrupt a continuous descent (Continuous Descent Operations, CDO). Both cost time and fuel. As an order of magnitude we use a **fuel hypothesis of 12.7 kg per minute of level flight**, derived from the EUROCONTROL ECAC-wide CDO study (46 kg of fuel for 217 s of level flight per flight from the top of descent, 2017 traffic), and $CO_2 = 3.16\times$ fuel. It is an average, not an aircraft performance model.

Focus airports. Paris-CDG (`LFPG`, large hub, four parallel runways), Toulouse-Blagnac (`LFBO`, medium airport, commercial traffic and Airbus operations) and Stavanger-Sola (`ENZV`, regional airport of UiS with offshore helicopter traffic), compared with two benchmark hubs: London-Heathrow (`EGLL`, two runways) and Frankfurt (`EDDF`, four runways).

Outside the scope. OPDI v0.0.2 has no scheduled times, so punctuality is out of scope: all delays are an **additional transit time** with respect to an empirical unimpeded reference, inspired by EUROCONTROL's ASMA indicator. Weather is not included; the results are associations, not causal effects.

2 Dataset: Source, Suitability, and Limitations

Source, volume and refresh.

1. **EUROCONTROL OPDI v0.0.2** (Parquet; monthly flight lists, ten-day windows of events and measurements; files can be republished):
   - `flight_list` (3,304,717 rows): flight id, origin and destination, aircraft type and class, operator, first/last seen times.
   - `flight_events` (86,670,133 rows): milestones of each flight (level-start/level-end, top of climb/descent, flight-level crossings, runway and parking events) with time, position and altitude.
   - `measurements` (118,779,222 rows): two cumulative values attached to events, `Distance flown (NM)` and `Time Passed (s)`.
2. **OurAirports** (CSV, updated daily): `airports` (86,350 rows) and `runways` (48,412 rows) for coordinates, elevation and runway identifiers.

In total 16 source files, i.e. 5.46 GB once stored in Bronze.

Suitability for distributed processing. The events and measurements (205 M rows together) must be filtered, windowed per flight and joined with the flights and the airport dimension. These are scans, shuffles and window functions over hundreds of millions of rows, a natural fit for Spark DataFrames and Spark SQL, run here in local mode on a VM with 4 vCPU and 7.8 GB of RAM.

Data quality risks (measured).

- **Missing airports:** the origin (`adep`) is missing for 43.5 % of the flights and the destination (`ades`) for 40.3 %; both are known for 38.5 %. The altitude is valid for 99.95 % of the events.
- **Uneven airport-event coverage:** only 48.8 % of the movement times come from an airport event (the rest from first/last seen times); the runway is known for 43.3 % of the movements (99.2 % of them matched to OurAirports). The runway entry at the destination is recorded for 82.8 % of the arrivals at Frankfurt and 83.4 % at Toulouse, but only 14.2 % at Heathrow and 13.9 % at Stavanger (Table 2).
- **Source gap:** the source has no movement at all in Europe on June 9, 2026 from 10:00 to 23:59 UTC (14 hours), plus one abnormally low hour (June 6, 19:00 UTC). The Gold regularity check flags these 15 incomplete hours (less than 50 % of the median traffic of the same hour); they are excluded from the daily indicators and from the ML training.

3 Bronze Layer: Ingestion and Preservation

Ingestion. `ingestion.py` asks the server for the size and `Last-Modified` date of each file (HTTP HEAD), downloads only new or republished files, and records every file in a manifest (URL, size, `Last-Modified`, SHA-256, run id, result). A disk guard (`min_free_gb = 4.0`) stops the downloads when the free space gets too low. In run `prod-4`, the republished OurAirports CSV files were detected and downloaded again.

Preservation and lineage. Bronze keeps the original columns and types, checked against a schema contract, and stores them in Parquet, partitioned by source period. Each row receives lineage columns: `_source_file`, `_source_period`, `_source_sha256`, `_source_last_modified`, `_ingestion_run_id`, `_ingested_at_utc`, `_bronze_run_id` and `_bronze_loaded_at_utc`. The lineage table is small and is

joined with a broadcast join. A reconciliation compares the row counts of each raw file with Bronze (16/16 files in `prod-8`).

Incremental Bronze. A state file (`bronze/_state.json`) stores the SHA-256 of the source file behind each Bronze partition. Only the files that are new or whose SHA-256 changed are rebuilt, with a dynamic partition overwrite; `-full-refresh` forces a complete rebuild. The state is saved only after a successful reconciliation.

```python
def files_to_build(settings, files):
    """Raw files whose Bronze copy is missing or comes from another version of the source file."""
    sha_now = {name: r["sha256"] for name, r in ingestion.last_manifest_entries(settings).items()}
    state = {} if settings.full_refresh else load_state(settings)
    todo = {}
    for table, paths in files.items():
        for path in paths:
            name = Path(path).name
            up_to_date = (sha_now.get(name) is not None and state.get(name) == sha_now[name]
                          and bronze_location(settings, table, name).exists())
            if not up_to_date:
                todo.setdefault(table, []).append(path)
    return todo
```

Listing 1: Selection of the files to rebuild in `bronze.py` (excerpt).

Evidence. The first full Bronze build took **525.7 s** (`prod-4`). In `prod-8` no source file had changed, so the Bronze tables step took **0.4 s**, with the reconciliation still passing on the 16 files.

4 Silver Layer: Data Model, Quality, and Performance (Primary Focus)

4.1 Data understanding and grain

- `silver/flights`: one row per flight (key `flight_id`), with standardised times, origin and destination enriched with OurAirports.
- `silver/flight_events`: one row per event (key `event_id`); every event references an existing flight.
- `silver/arrivals`: one row per arrival with a known destination (1,866,643 rows): 100 NM ring entry (time, position, sector), runway entry, transit time, reference and additional time, level-off and holding summary.
- `silver/descent_level_offs`: one row per fused level-off during the descent (1,919,728 rows).

The exploration of the real data shaped these definitions: OPDI splits one level flight into many short level-start/level-end pairs about 10 s apart (hence the fusion below), level events also exist on the ground for parked aircraft (excluded), OPDI's top-of-descent is missing for about one third of the arrivals (not used as a boundary), and the measurements are never attached to runway events.

4.2 Cleaning, standardisation, and quarantine

Silver converts the types and gives one form to missing values, removes duplicates, and applies quality rules. A row that fails a rule is not dropped silently: it goes to a quarantine table with the first rule it fails (e.g. `DURATION_GT_24H`, `INVALID_COORDINATES`, `ORPHAN_EVENT`). In `prod-8`, 625 flights (`DURATION_GT_24H`) and 9,100 events (`ORPHAN_EVENT`, events of a flight that is not valid) were quarantined. A before/after report proves the row accounting: Bronze = out of scope + quarantined + duplicates + Silver (Table 1).

Table 1: Row accounting across the layers (run `prod-8`).

Layer	Table	Input rows	Output rows	Accounting
Bronze	`flight_list`	3,304,717	3,304,717	reconciled with the raw files
Bronze	`flight_events`	86,670,133	86,670,133	reconciled with the raw files
Bronze	`measurements`	118,779,222	118,779,222	reconciled with the raw files
Silver	`flights`	3,304,717	**3,127,830**	−175,452 out of scope, −810 duplicates, −625 quarantined
Silver	`flight_events`	86,670,133	**84,000,238**	−2,660,795 out of scope, −9,100 quarantined
Silver	`arrivals`	3,127,830	**1,866,643**	flights with a known destination
Silver	`descent_level_offs`	—	**1,919,728**	fused level-offs
Gold	airport movements	3,127,830	**3,636,308**	1,767,608 departures, 1,868,700 arrivals

4.3 Arrival-efficiency indicators in PySpark

Indicator A — additional arrival time. For each arrival, the entry into the 100 NM circle around the destination is interpolated between the last event outside and the first event inside the circle (d: distance to the airport):

$$t_{\text{entry}} = t_1 + \alpha\,(t_2 - t_1), \qquad \alpha = \frac{d_1 - 100}{d_1 - d_2} \tag{1}$$

The entry sector is one of 8 sectors of 45°. The runway time is the first `entry-runway` event at the destination below its elevation + 1,500 ft. The **unimpeded reference** T_{ref} is the 20th percentile of the transit time per airport × runway × aircraft class × entry sector, with a fallback to class × sector, then to the airport, when a group has fewer than 20 flights:

$$\Delta t = (t_{\text{runway}} - t_{\text{entry}}) - T_{\text{ref}}, \qquad 5 \le t_{\text{runway}} - t_{\text{entry}} \le 120 \text{ min} \tag{2}$$

Indicator B — level-offs and holdings. Within 200 NM of the destination, from the last event at the highest level to the runway, consecutive level chunks less than 60 s and 200 ft apart are fused with window functions; a fused level-off is kept if it lasts at least 30 s and is at least 1,000 ft above the airport. Level-offs are grouped in four bands (below FL70, FL70–100, FL100–200, FL200 and above). A **holding** is a level-off of at least 4 min whose distance flown (from the measurements) is at least 1.5 times its displacement.

```
w_merge = Window.partitionBy("flight_id").orderBy("seg_start_utc")
with_grouping = (
    paired_segments
    .withColumn("prev_end_utc", lag("seg_end_utc").over(w_merge))
    .withColumn("prev_alt_ft", lag("seg_alt_ft").over(w_merge))
    .withColumn("gap_s", unix_timestamp("seg_start_utc") - unix_timestamp("prev_end_utc"))
    .withColumn("alt_diff_ft", spark_abs(col("seg_alt_ft") - col("prev_alt_ft")))
    .withColumn("is_new_group", when(
        col("prev_end_utc").isNull() | (col("gap_s") < 0)
        | (col("gap_s") > cfg.level_merge_max_gap_s)
        | (col("alt_diff_ft") > cfg.level_merge_max_alt_diff_ft), lit(1)).otherwise(lit(0)))
    .withColumn("group_id", spark_sum("is_new_group").over(w_merge))
)
# then groupBy(flight_id, group_id): start = min, end = max, altitude = median,
# first/last position and distance flown = min/max of a struct ordered by time (deterministic)
```

Listing 2: Fusion of the level chunks in `arrivals.py` (excerpt).

4.4 Performance: evidence and tuning

Where the time goes. The final production run `prod-8` took 14 min 43 s for the deployment job (ingest 36 s, Bronze 19 s, Silver 10 min 35 s, Gold 1 min 8 s, ML 1 min 2 s), against 18 min 44 s for `prod-4` (first production run, before the arrival study), mainly thanks to the incremental Bronze. The slowest steps of `prod-8` are writing `silver/flight_events` (263.9 s), the arrival study (215.7 s, of which 153.3 s for the level-offs and 55.8 s for the arrivals) and writing `quarantine/flight_events` (104.7 s).

Incident `prod-5`. The first version of the arrival study received the *lazy* Silver DataFrames, so Spark recomputed the whole Silver chain (86 M events) several times, and the run failed after 13 minutes with `java.lang.OutOfMemoryError: GC overhead limit exceeded`. The fix (PR #12):

1. read the *written* Silver tables instead of the lazy DataFrames;
2. read only the needed columns, and keep only the events within 300 NM of the destination, cached on disk;
3. join a single measurement type (`Distance flown (NM)`) instead of aggregating the 119 M measurements;
4. write `silver/descent_level_offs` first, then build `silver/arrivals` from it.

The next run (`prod-6`) passed with an arrival step of 217 s; it took 215.7 s in `prod-8`.

Other choices and what was not tuned. Small tables (lineage, airport dimension) are joined with broadcast joins, and the events are partitioned by source period. The quarantine write recomputes the event rules for only 9,100 rows (about 100 s): identified, but not optimised, since a full production run takes about 14 minutes for a monthly schedule; caching the flagged events (and a 4 GB driver) would be the next steps if the volume grew.

Scalability. The Silver cost grows linearly with the number of events. Late or republished source files only trigger the rebuild of their own Bronze partitions; a schema change in a source file fails the Bronze schema contract before any processing.

5 Gold Layer: Serving the Outcome

Gold is written in **Spark SQL** (one query per table in `src/opdi_pipeline/sql/`) and differs from Silver: smaller tables, aggregated for one question each.

- **Traffic:** `airport_daily_ops` (43,397 rows), `airport_hourly_ops` (474,130), `runway_usage` (23,500), `airport_ground_times` (43,397).
- **Arrival efficiency:** `arrival_reference` (3,046), `airport_arrival_daily` (41,334), `airport_arrival_hourly` (381,863), `holding_hotspots` (20,438).
- **Correctness and readiness:** 30 validation checks (reconciliation with Silver, ranges, consistency, schema contracts) all passed in `prod-8`. Gold is published (`_SUCCESS` marker and a `run_status` history) only if every check passes; otherwise the marker is removed and the run fails. The regularity check flags the incomplete hours so that the consumers can exclude them.

6 Machine Learning

6.1 Explanatory model

A Spark MLlib `GBTRegressor` (20 iterations, depth 5) explains the additional time of each measured arrival in complete hours.

- **Features:** airport, entry sector, runway, aircraft class, operator (top 30), hour, day of week, hourly demand, previous-hour demand, and load factor (demand / airport's 90th-percentile demand). The number of level-offs is not a feature: it is part of the delay being explained.
- **Temporal split:** training before 2026-07-14 (534,683 arrivals), test on the last 14 days (200,937 arrivals).
- **Quality gate:** the model must beat a naive baseline (mean per airport and hour). Model: MAE 2.74 min, RMSE 4.44, R^2 0.135; baseline: MAE 2.84, RMSE 4.64, R^2 0.053. The model was therefore published. Per focus airport, however, the gain is nil (LFPG 2.65 vs 2.66, EDDF 2.53 vs 2.56, LFBO 1.87 vs 1.86) or negative (EGLL 4.28 vs 4.10).

6.2 Variable importance and demand-smoothing scenario

The variable importance (Figure 7, left) is dominated by the destination airport (0.56), then the aircraft class (0.16), the runway (0.08), the hour (0.07) and the load factor (0.05). These are importances in the trees, not shares of explained variance: the model explains 13.5 % of the variance.

The scenario caps the hourly demand at the airport's 90th percentile and moves the excess flights to the nearest hours with room (same day, test period). Predicted savings: 7.0 min/day at Frankfurt (cap 51/h, 7 flights moved per day), 6.8 at Toulouse (cap 8/h), 1.8 at Paris-CDG (cap 35/h, 19 moved per day), 0 at Stavanger and −0.5 at Heathrow (cap 45/h). The negative value at Heathrow shows the limit of one global model for more than 600 airports with only 14 % of Heathrow's arrivals measured: it does not capture the demand effect visible in the descriptive data (Figure 4), which remains the stronger evidence.

6.3 Airport typology

A `StandardScaler` + `KMeans` clustering, with k chosen by the silhouette among 3–6, groups 119 representative airports (at least 100 measured arrivals and 30 % coverage) into $k = 3$ clusters (silhouette 0.55; Figure 2, right): "large airports, fluid arrivals" (23, e.g. EDDF, EHAM, LEMD), "medium airports, fluid arrivals" (77, e.g. LFBO) and "small airports, high additional time and frequent holdings" (19). EGLL and LFPG are not clustered (coverage below 30 %).

7 Results at the Focus Airports

Table 2: Arrival efficiency at the focus airports (`prod-8`, 58 days, complete hours).

	LFBO	EDDF	LFPG	EGLL	ENZV
Arrivals	6,265	37,768	29,504	38,004	4,264
100 NM ring entry	86.3 %	97.1 %	98.1 %	97.1 %	47.7 %
Runway entry	83.4 %	82.8 %	26.4 %	14.2 %	13.9 %
Measured transit (A)	77.7 %	81.8 %	26.2 %	13.8 %	1.1 % (48)
Additional time (A)	**1.9 min**	**2.7 min**	**3.1 min**	**4.9 min**	n/a
Level-off min / arrival (B)	**1.6**	**4.1**	**4.6**	**4.4**	**0.2**
below FL70	0.72	1.36	1.92	0.25	0.08
FL70–100	0.09	0.35	0.31	2.27	0.04
FL100–200	0.12	0.96	1.06	1.24	0.02
FL200 and above	0.64	1.46	1.35	0.63	0.09
Arrivals with holding	**0.7 %**	**2.9 %**	**2.9 %**	**12.7 %**	**0.2 %**
Holding minutes (58 days)	415	6,356	5,812	32,802	214
Fuel hypothesis (level flight)	2.2 t/day	34 t/day	30 t/day	37 t/day	0.2 t/day

Level-off minutes per arrival by band: sum over the 58 days of the Gold table `airport_arrival_daily` divided by the arrivals.

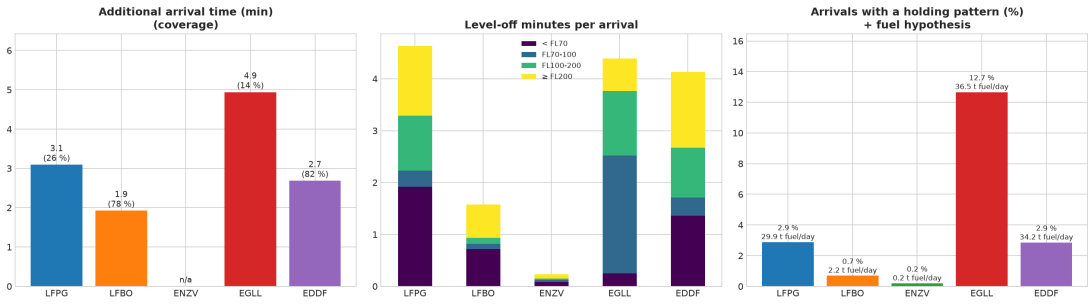


Figure 1: Focus airports: additional arrival time with its coverage (left), level-off minutes per arrival by flight-level band (centre), share of arrivals with a holding and fuel hypothesis (right).

London-Heathrow — demand-driven delay. 12.7 % of the arrivals hold (32,802 holding minutes over 58 days), and the level-offs are concentrated between FL70 and FL100 (2.27 min per arrival) and FL100–200 (1.24 min). The holdings form four zones, one per quadrant around the airport (Figure 3), consistent with Heathrow's four holding stacks. The additional time grows with the demand: about 0.1–0.2 min below 20 arrivals per hour, 2.4 min at 20–29, 4.5 min at 30–39 and 6.8 min at 40–49 (Figure 4). The worst hours are the morning wave (07–08 UTC, about 8–10 min; Figure 5).

Paris-CDG and Frankfurt — level flight rather than holding. Both airports rarely hold (2.9 %) but fly 4.6 min (CDG) and 4.1 min (Frankfurt) level per arrival. At CDG the level-offs are below FL70 (1.92 min), between FL100 and FL200 (1.06 min) and above FL200 (1.35 min); its holdings are mostly east of the airport. Under the 12.7 kg/min hypothesis this level flight represents about 30 t (CDG) and 34 t (Frankfurt) of fuel per day; not all of it is avoidable. Their additional time does not grow with the demand: 2.5–2.9 min at Frankfurt between 20 and 59 arrivals per hour (it only rises in the few hours above 60), 2.4–4.3 min at CDG with no clear trend; CDG's worst hour is 06 UTC (4.9 min). Figure 6 shows real descents with and without holding at Heathrow and CDG.

Toulouse and Stavanger. Toulouse is fluid: 1.9 min of additional time (1.8–2.3 min whatever the demand), 1.6 min of level flight and 0.7 % of arrivals with a holding. Stavanger shows only 0.2 min of level flight and 0.2 % of holdings, but its Indicator A cannot be measured (48 measured arrivals) and less than half of its arrivals are even seen entering the 100 NM ring: its low values partly reflect the poor receiver coverage, and must be read with care.

Europe-wide. Among the airports with at least 10,000 arrivals and 50 % coverage (mean of the daily averages), the highest additional times are at London-Gatwick (4.3 min), Lisbon (3.9) and Milan-Malpensa (3.7), the lowest at Stockholm (2.0) and Budapest (1.9) (computed in run `prod-6`, whose Gold arrival tables are the same as in `prod-8`; Figure 2).

Recommendations (hedged). (1) Heathrow: the dependence on demand points to arrival flow management (spreading the morning wave, sequencing before the holding stacks). (2) Paris-CDG and Frankfurt: little holding but 4–5 minutes of level flight per arrival point to continuous-descent procedures. (3) Toulouse and Stavanger: no significant gain to expect on the arrival phase.

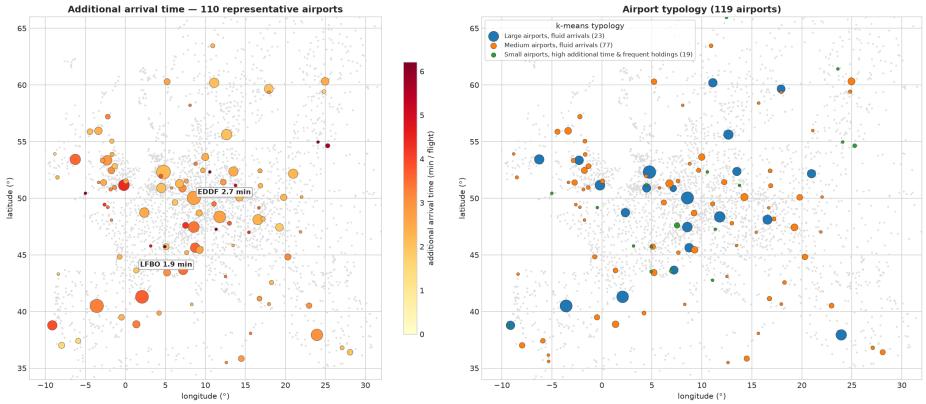


Figure 2: Additional arrival time at 110 representative airports (left) and KMeans typology of 119 airports (right).

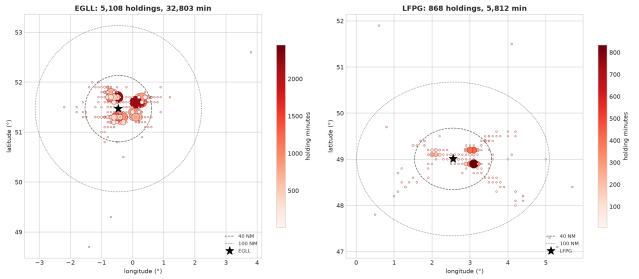


Figure 3: Location of the holdings around Paris-CDG and London-Heathrow, with the 40 NM and 100 NM rings.

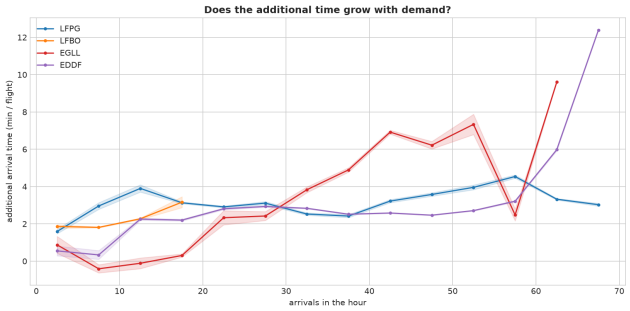


Figure 4: Mean additional arrival time by number of arrivals in the hour (bins of 5 arrivals per hour).

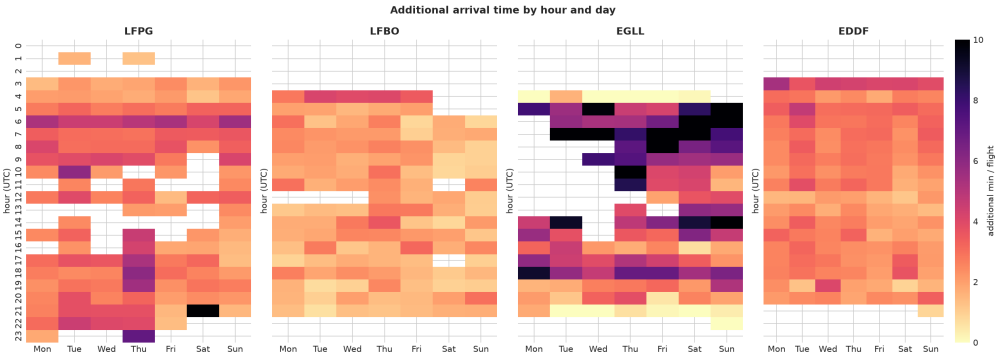


Figure 5: Mean additional arrival time by hour (UTC) and day of the week.

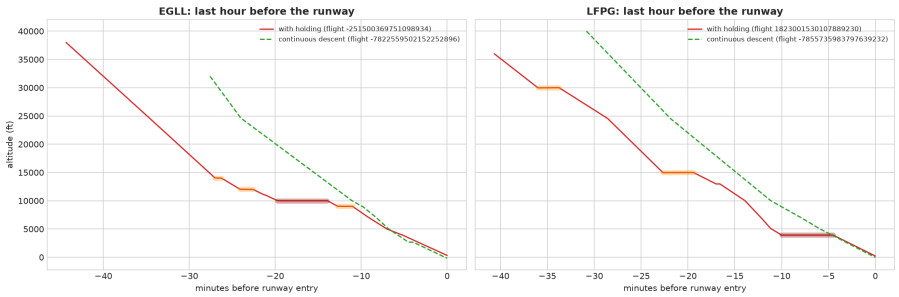


Figure 6: Real descent profiles (last hour before the runway): one flight with holding (level-offs highlighted) and one continuous descent, at Heathrow and Paris-CDG.

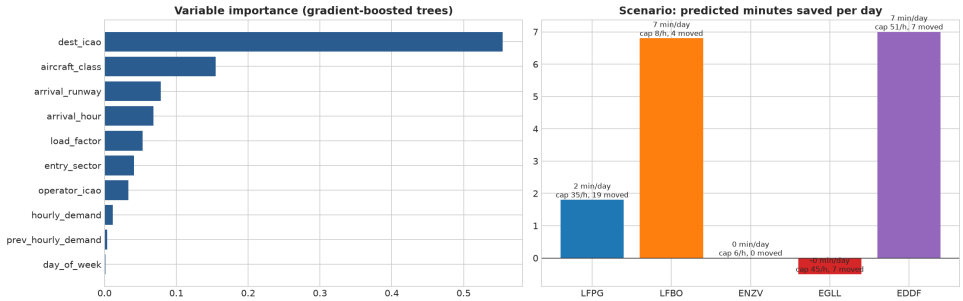


Figure 7: Variable importance of the gradient-boosted trees (left) and predicted minutes saved per day by the demand-smoothing scenario (right).

8 End-to-End Delivery and Operations

Reproducibility and environments. The code is a Python package with one module per layer (`src/opdi_pipeline/`), run by a command-line entry point, `run_pipeline.py`, with the options `-env` (`dev` or `prod`) and `-layer`. Two configurations: `config/dev.toml` (10-day sample, June 9–19, data in `~/opdi-data-dev`) and `config/prod.toml` (full period, `~/opdi-data`). Run instructions: README.

Tests (46 pytest tests). Unit tests of the configuration, ingestion, Silver rules, arrival rules, ML functions and schemas; a static check that the notebooks only call existing package functions; and an end-to-end test of every layer through the CLI on a synthetic OPDI dataset served by a fake EUROCONTROL server, with planted defects (duplicates, orphan events, holdings, ground level-offs) and exact expected values.

CI/CD with GitHub Actions. A feature branch is merged into `dev` by pull request: the tests run on a GitHub-hosted runner, then the whole pipeline runs on the dev sample on the VM through a self-hosted runner. A pull request from `dev` to `main` triggers the production run on the VM after a **manual approval** (GitHub environment `main`). Schedules: nightly dev run, monthly production run. On-demand jobs: data exploration and notebook execution. Each run uploads its logs, JSON reports and Gold tables as artifacts.

Observability and quality gates. Every run writes JSON reports and a `pipeline_summary.json`. Seven quality gates must pass (all passed in `prod-8`): Bronze schema contract, Bronze reconciliation, Silver row accounting, no orphan event in Silver, Silver schema contracts, Gold validation, Gold published.

9 Reflection and Limitations

9.1 What worked, what did not

- **Worked:** the incremental Bronze (525.7 s → 0.4 s when nothing changed); writing intermediate tables and reading them back instead of chaining lazy DataFrames; pruning columns and events early; the quality gates and run reports, which make the state of every run visible; the end-to-end test on synthetic data, run on every pull request before anything reaches the VM.
- **Did not work at first:** passing lazy DataFrames between the Silver steps and the arrival study (`prod-5` out of memory). Lesson: on this volume, materialise the large intermediate results and measure before tuning.
- **To improve:** per-airport models and weather data, caching the flagged events for the quarantine write, and a wider period.

9.2 Limitations

No scheduled times (no punctuality, only additional time over an empirical reference); no weather, so part of the variance is unexplained and the results are associations, not causality; partial and uneven runway coverage (Table 2); a percentile reference cannot see a delay suffered by almost every flight of a group; the fuel figures rest on an average hypothesis (12.7 kg/min), not an aircraft performance model; 14-hour source gap on June 9; OPDI v0.0.2 is a beta product.

References

1. EUROCONTROL. *Open Performance Data Initiative (OPDI)*, v0.0.2. https://www.eurocontrol.int/performance/data/
2. OurAirports. *Open data downloads* (airports, runways). https://ourairports.com/data/
3. EUROCONTROL. *Continuous climb and descent operations* (CDO study, 2017 traffic). https://www.eurocontrol.int/concept/continuous-climb-and-descent-operations
4. Group 08 repository: https://github.com/mathis6938/group08. Production run `prod-8` (2026-11-09).